

Bias and variance

⊗ Bias: → Refers errors due to overly simplistic assumption made by the model.

→ Model with high bias tends to underfit the data.

→ perform poorly in both training and testing

⊗ variance: → Refers errors due to model being more complex and sensitive to specific training data it has seen.

→ Model with high variance tends to overfit the data.

→ performs very well on training but poorly in test data.

Bias-variance tradeoff: Helps to balance complexity and the simplicity of the model.

i) High bias, low variance $\rightarrow$ underfitting

ii) low bias, high variance $\rightarrow$ overfitting

Trade off visualisation:

$$\text{Total error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible error}$$

Boosting

Bagging - Reduces variance by training multiple models independently. (Reduces overfitting)

Boosting - Reduces both bias & variance by training the model sequentially. Each model tries to correct the errors made by previous model. This creates strong model from multiple weak models.

How it works ?

- Models are trained sequentially with each new model giving more importance to the data points that previous models misclassified.
- The predictions from all the models are combined, usually by a weighted sum / vote, where more weight is given to the models that performs better.

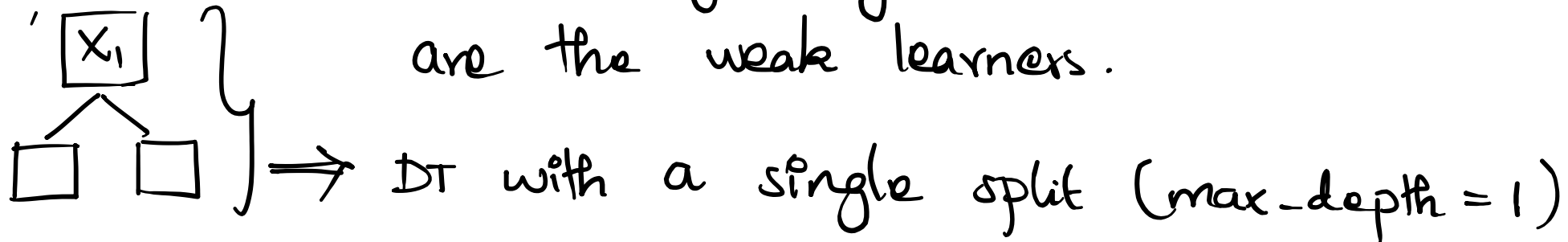
Examples:

1. AdaBoost (Adaptive Boosting)
2. Gradient Boosting
3. Extreme Gradient Boosting (XGBoost)

Adaboost: Ensemble learning algorithm designed to improve the performance of weak learners by combining them into stronger overall model.

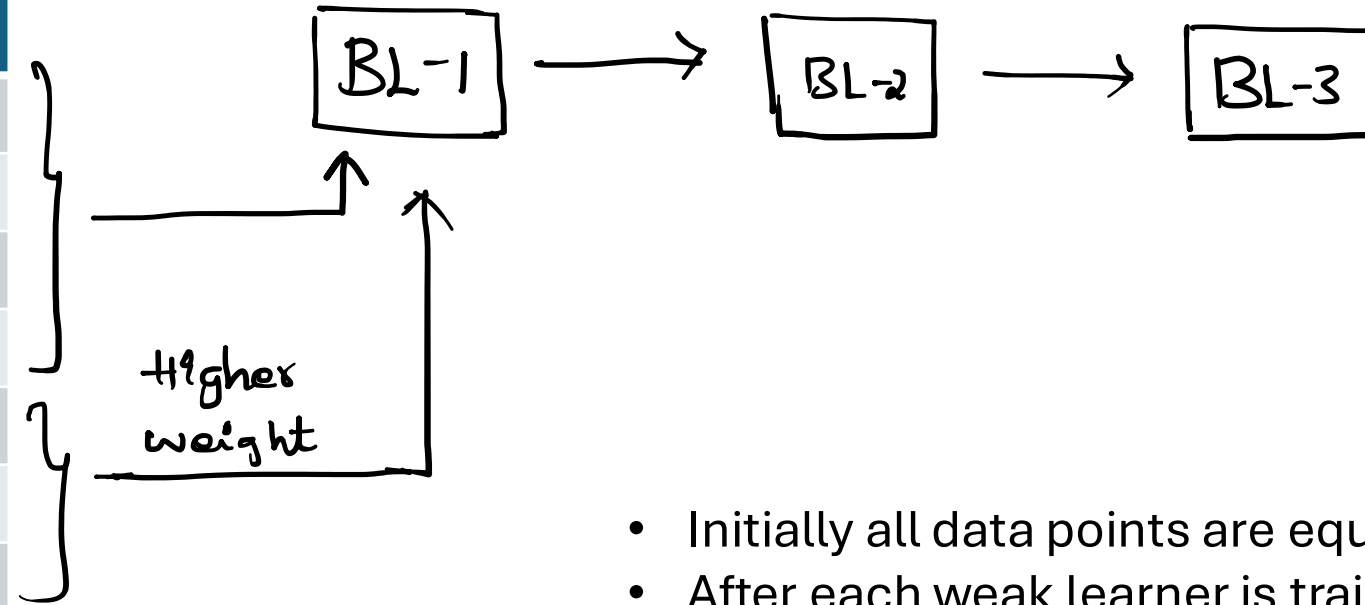
④ Focus on misclassified datapoints on iterative training and adjust the mistake made by previous model.

Weak learners: models that perform slightly better than random guessing. In adaboost decision stumps are the weak learners.



Sequential Training:

X1	X2	X3	Y
1			✓
2			✓
3			✓
.			✓
.			✗
.			✗
1000			✗



- Initially all data points are equally weighted.
- After each weak learner is trained Ada boost increase the weight of misclassified samples and decrease the weights of correctly classified.
- This forces the next weak learner to pay more attention to the hard to classify points.

How Adaboost works:

- i) Initialization - Assign equal weights to all dp's.
If we have N samples, weight of each sample = $1/N$
- ii) Training the weak learners.
- iii) update the weights of records.
- iv) Repeat (ii) & (iii) till predefined iteration.
- v) Final model with aggregates of votes for classification or weighted outputs for regression.

④ Error Rate of weak learner:

$$\Rightarrow E_t = \sum_{i=1}^n w_i \times I(y_i \neq h_t(x_i))$$

Where,

w_i - weights of samples i .

y_i - true label.

$h_t(x_i)$ - prediction from weak learner.

$I(y_i \neq h_t(x_i))$ - Indicator function that equal to 1 if the prediction is correct and 0 otherwise.

④ Learner weight (α_t): Performance

④ contribution of each weak learner to final prediction

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

④ Weight update:

$$\Rightarrow w_i(t+1) = w_i(t) \cdot e^{\alpha_t \times I(y_i \neq h_t(x_i))}$$

④ If the prediction is correct in previous iteration weight will reduce in next iteration & vice versa.

X1	X2	X3	Y
1			
2			
3			
4			
5			
6			X
7			

Sample Weight

$1/7$

.

.

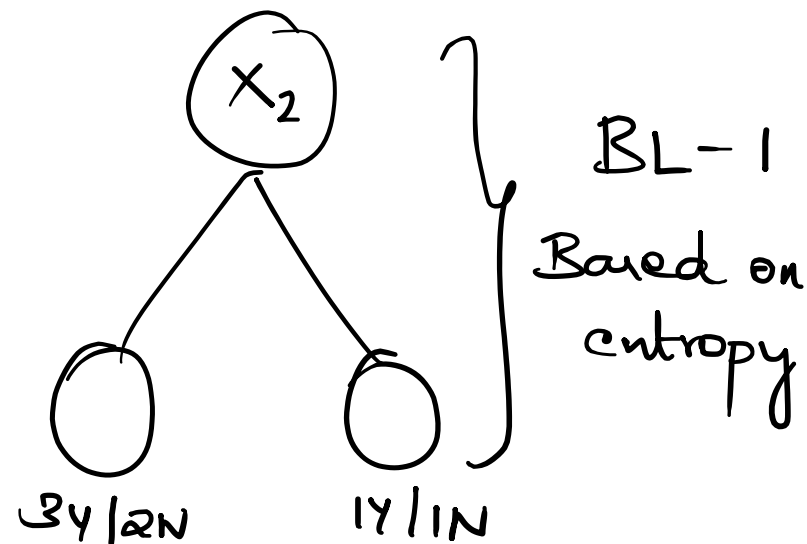
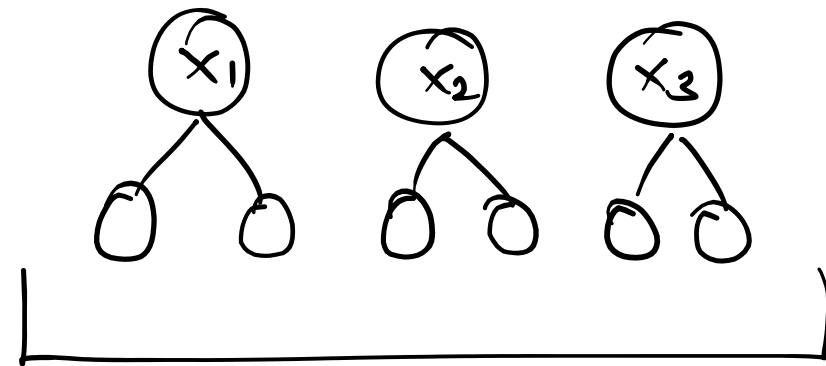
.

.

.

$1/7$

Decision stumps



Performance of decision stump (BL-1):

$$\Rightarrow \frac{1}{2} \log_e \left(\frac{(1 - \epsilon_t)}{\epsilon_t} \right)$$

$$\Rightarrow \frac{1}{2} \log_e \left(\frac{(1 - 1/7)}{(1/7)} \right) \Rightarrow 0.89$$

Weight update :

$$\begin{aligned}\Rightarrow w_i(t+1) &= w_i(t) \cdot e^{\alpha_t} \\ &= \frac{1}{7} \times e^{0.89} \\ &= 0.35\end{aligned}$$

$$\begin{aligned}&= w_i(t) \cdot e^{-\alpha_t} \\ &= \frac{1}{7} \times e^{-0.89} \\ &= 0.05\end{aligned}$$

$$\frac{0.35}{0.65} \Rightarrow$$

X1	X2	X3	Y
1			
2			
3			
4			
5			
6			X
7			

Sample Weight

$\frac{1}{7}$

.

.

.

.

.

$\frac{1}{7}$

updated Weight

0.05

0.05

0.05

0.05

0.05

0.35

0.05

Normalized

0.076

0.076

0.076

0.076

0.076

0.538

0.076

X1	X2	X3	Y	Weights
1				1/7
2				1/7
3				1/7
4				1/7
5				1/7
6			×	1/7
7				1/7

Normalized Weights	Bucket
0.07	0 to 0.07
0.07	0.07 to 0.14
0.07	0.14 to 0.21
0.07	0.21 to 0.28
0.07	0.28 to 0.35
0.53	0.35 to 0.88
0.07	0.88 to 0.95